

阅读目录

Paper2 实验导读：哪些实验完成了，分别证明了什么	2
先找到论文里的实验	2
1. 主实验到底用了什么 benchmark	2
2. 看懂三个容易混淆的指标	3
3. 奖励审计：为什么旧模型不做关系恢复	3
4. 主 RL 对比：改奖励后怎样，是否胜过简单方法	3
5. 消融：拿掉组件后会怎样	4
6. 图规模：图变大后会怎样	4
7. 双策略生成：更多候选是否值得更多调用	5
8. 真实候选执行：为什么不是“规则覆盖 100%”	5
9. DocRED 开发实验：有来源文本和类型后如何	5
10. 新 20 文档：来源规则有作用，但会误删	6
11. 最新核验：完成了哪些，还缺哪些	6
12. 其他辅助实验放在哪里	7
13. 训练、调用和统计如何计数	7
14. 用这张表记住论文结论	7

Paper2 实验导读：哪些实验完成了，分别证明了什么

更新：2026-10-10。按当前论文顺序整理，不要求先读懂强化学习公式。本轮论文使用简单英文与直接表达，导读同步新章节名；没有新增实验或 API 调用。

这篇论文想回答两件事：**怎样安排找规则与修图的顺序？两种生成方式能否提供真正有用的规则？** 所以读结果时要把“调度算法表现”和“生成规则质量”分开。

先找到论文里的实验

当前 TKDE 模板中 III 是方法章，IV 是实验章；IV-B 表示实验章第二小节。以英文小节名定位可以避免云端改版造成编号差异。

本导读	论文位置	回答的问题
1—2	IV-A Test Design	主 benchmark 是什么，分数是什么意思？
3	IV-B / Checking the Reward	旧策略为什么不愿修某些错误？
4	IV-B / Policies and Scores、Policy Results	修正奖励后，学习策略表现如何？
5	IV-B / Component Tests with the Rate Reward	哪些组件有独立作用？
6	IV-B / Fixed Policies on Larger Graphs	图变大时会怎样？
7	IV-C Candidates from Two Prompts	两种策略是否更互补、更划算？
8	IV-D / Direct Execution of Saved Patterns、Generated Rules in a Repair Loop	生成的规则能否实际修图？
9—10	IV-D / Document Development Tests; 附录 C、D	换成有类型的真实文档后如何？
11	附录 D-A Rule Approval and Coverage	核验规则能否避免误删？
12	附录 F、H、I	历史消融和其他辅助检查说明什么？
13	IV-E Time and Cost; 附录 E、G	训练、调用和重复回放怎样计数？
14	IV-F Discussion; Conclusion	目前完整结论是什么？

具体公式见 [数学导读](#)。这里先讲实验问题和答案。

1. 主实验到底用了什么 benchmark

对应论文：IV-A, [experiments.tex](#), subsubsec:testset。

主 RL 实验使用的是 **TNEWS/CLUE 新闻构造的自定义受控图修复任务**，不是直接采用一个现成的通用 KG 修复排行榜。

做法是：选 450 条新闻，15 类各 30 条；按类别和关键词确定性地建图，得到 873 个节点、1,083 条边；再放入孤立、重复、非法关系和悬空边四类缺陷。

策略可以获取固定检查模块或执行修复。获取模块不调用大模型；关系恢复使用环境保存的损坏前记录，策略本身看不到这份答案。

还有三类不同的数据，不要混成主实验：

数据	规模	用途
规则生成档案	4,728 文档，两策略共 9,456 个成功响应	比较候选数量和重合
RuleTest-94	30 个干净项+64 个设计缺陷	检查规则能否匹配并执行
DocRED 文档开发研究	从原 train_annotated 内部分成 180/60/60	检验实际生成规则、来源证据和四动作执行

DocRED 的内部 test 不是官方测试集；当前新生成规则的正式学习和预留测试还没做。TNEWS 的 20 个开发场景、30 个预留测试场景也共享同一基础图，只是损坏方式不同，不代表跨文档或跨领域测试。

2. 看懂三个容易混淆的指标

对应论文： IV-B 的 Policies and Scores, [experiments.tex](#)。

指标	简单含义	要注意什么
结构质量 Q	孤立、重复、非法关系、悬空等问题少不少	高分不等于事实都正确
参考三元组 F1	最终输出与参考事实有多接近	同时惩罚漏项和额外项
原非法关系恢复率	原先那条坏边是否还在，而且关系修对了	直接删掉坏边不能得恢复分

例如一条坏边被删除，结构分可能变好，但它表达的信息也没了，所以不能算“关系恢复”。

过程 AUC 则看多步操作过程的平均质量。它不是分类 ROC-AUC，也不是最终 F1。论文需要同时报告这些量，防止只看一个漂亮分数。

3. 奖励审计：为什么旧模型不做关系恢复

对应论文： IV-B 的 Checking the Reward, 图 fig:reward_action_audit。

旧 count 惩罚下，DDQN 没有引入新违规，看起来安全，但它实际一次关系恢复也没做。检查旧轨迹中的 155 个可执行关系修复分支发现：这些动作在旧奖励下全部得到负的即时回报。

原因是修对关系可能暂时形成重复边，后面还可以去重；旧奖励却对每个重复扣固定分，把有用修复也罚成了不值得做。

同样的分支，平均即时回报从 count 的 -0.01990 变成 rate 的 $+0.00821$ 。

结论： 旧策略不修，不一定是不会修，也可能是奖励在阻止它修。这里是对旧状态分支的诊断，155 个分支不是 155 篇独立测试文档。

4. 主 RL 对比：改奖励后怎样，是否胜过简单方法

对应论文： IV-B 的 Policies and Scores、Policy Results；表 [reward_validation.tex](#)。

DDQN 是论文训练的动作选择网络，DQN 是其标准版本。Ridge 是简单的线性预测器，预测当前动作的收益；启发式则按预先写好的选择规则工作。种子是控制随机初始化和采样的编号，多种子用于观察结果是否稳定。

四种神经设置各训练 10 个种子、每个 250 回合，共 40 个模型；另有 10 个简单 ridge 预测器，以及固定启发式与随机策略。模型封存后，在 30 个预留损坏场景上比较。

表中的 count/rate/zero 是新违规惩罚的三种算法：按数量扣、按初始规模占比扣、不扣。其他关键条件保持匹配。

策略	参考 F1	原非法关系恢复率	模拟获取成本/图
DDQN-count	94.67%	0.00%	6.27
DDQN-rate	99.49%	94.00%	4.42
DDQN-zero	99.75%	99.06%	4.06
DQN-rate	99.77%	96.94%	4.21
简单 one-step ridge	99.88%	97.90%	4.04
先获取可靠规则，再修最大缺陷的启发式	99.91%	98.77%	4.00

策略	参考 F1	原非法关系恢复率	模拟获取成本/图
Rule-first valid	99.40%	94.73%	8.00
Random valid	97.84%	70.72%	9.72

确定的结论：rate 比旧 count 改善了约 4.82 个百分点 F1，也恢复了有用修复行为，预设检验支持这个差异。

没有证明的结论：DDQN 比最强简单方法更好。启发式和 ridge 的平均 F1 仍更高；相关差异未通过校正，也不能反过来说两者已经等效。

所有策略的初始正确参考保留率都是 100%，与环境使用可信恢复记录有关。不能据此推断真实生成规则也不会误删。

5. 消融：拿掉组件后会怎样

对应论文：IV-B 的 Component Tests with the Rate Reward, [rate_ablation.tex](#), 表 tab : rate _ablation。

在当前 rate 奖励下，为五种改动分别重新训练 10 个模型，共新增 50 个模型。使用另 30 个扰动场景，共 2,100 次策略/种子/场景评估。

设置	F1	关系恢复率
完整 rate DDQN	99.52%	93.77%
去图特征	99.41%	89.38%
去规则特征	97.75%	66.82%
去动作掩码	97.02%	57.76%
去调用惩罚	98.54%	76.86%
小系数 count 惩罚	99.05%	92.14%
强启发式	99.91%	98.84%

动作掩码是“当前允许做哪些动作”的清单。去掉后 F1 下降 2.50 点，并频繁选择不可执行动作；它是本组十二项比较中唯一通过多重比较校正的对照。

结论：掩码有较明确的实验证据；不能把其他均值下降都写成组件显著必要。特征置零时，其他特征和掩码还保留部分信息，也不等于完全移除该类知识。

小系数 count 与 rate 没有显著差别，说明 rate 的改善不一定全部来自“按规模归一化”，也可能部分来自惩罚变轻。这里的 99.52% 与主表 99.49% 来自不同扰动集合。

6. 图规模：图变大后会怎样

对应论文：IV-B 的 Fixed Policies on Larger Graphs, [scale_control.tex](#)。

把基础图复制为 1、2、4、8 份，彼此不连接；最大 6,984 节点、8,664 条边。用已经训练好的策略评估，共 800 个结果，没有在每个规模重新训练。

单独测量启发式的运行资源：基础规模约 0.881 秒，8 倍规模约 8.805 秒；新增 Python 分配峰值约从 1.32 到 11.35 MiB。

结论：可以观察重复图规模增长后的耗时及固定策略表现，但没有验证真实大型连通图。这里的内存也不是整个进程的全部占用。

7. 双策略生成：更多候选是否值得更多调用

对应论文：IV-C, `experiments.tex`, 表 `offline_budget.tex`、图 `fig:rule_quantity`。

同一批 4,728 文档, `deletion` 和 `augmentation` 各生成一次:

策略	去重后的分类候选数	成功调用数
Deletion	32,955	4,728
Augmentation	41,683	4,728
两者合并	71,656	9,456

这些总数还包括实体和关系声明, 不全是可执行约束。合并比单用 `augmentation` 多 71.9%, 但也用了两倍调用。

再按相同调用预算比较: 1,000 次调用时, `augmentation` 平均产生 7,787 个不同约束候选, 双策略是 7,126。其他测试预算也没有出现双策略的数量优势。

结论: 两种策略在同一文档集上能提出不同内容; 尚未证明每次调用更划算。更多候选还要经过下一步的实际执行检查。

8. 真实候选执行：为什么不是“规则覆盖 100%”

对应论文：IV-D 的 `Direct Execution of Saved Patterns`、`Generated Rules in a Repair Loop`; 附录 B、I。

历史候选中, 精确类型模式能编译成 18,143 条不同规则。但在 `RuleTest-94` 上只有两种模式匹配到记录。

真正执行生成规则	移除设计缺陷	误删干净项
Deletion	10/64	0/30
Augmentation	0/64	0/30
合并	10/64	0/30

结论: 规则多不等于覆盖广。这里缺陷召回为 15.6%, `augmentation` 没有提供额外修复; 四种获取/修复顺序的最终结果相同。

另一个手写规则族测试能覆盖 64/64, 放在附录 H-A。它是人工实现检查器在设计用例上的回归测试, 不能替换生成候选的 10/64 成绩。

三份真实图快照还暴露出另一问题: 26,270 条边缺少端点类型, 规则无法可靠匹配, 程序选择不处理。这不表示那些图没有错误。

9. DocRED 开发实验：有来源文本和类型后如何

对应论文：IV-D 的 `Document Development Tests`; 附录 C、C-A, `docred_source_pilot.tex`。

DocRED 提供原文和给定实体类型, 减少“图里没类型”的问题。新环境每次获取一份真实响应, 选择获取 `deletion`、获取 `augmentation`、修复或停止。

开发阶段	主要结果	说明
最初有类型的 pilot	生成规则, 但没有触发删除	规则存在不代表可用
旧第一轮, 20 文档	F1 94.42%→94.59%, 损失 4 条参考事实	改善小, 同时误删

开发阶段	主要结果	说明
旧第二轮，同样 20 文档	F1 95.69%，移除 7/31 注入项，无参考损失	全部来自 deletion 类型规则，来源矛盾规则没有发挥作用

把旧两轮所有可行操作顺序枚举出来，共 598 条路径。第二轮即使用参考答案挑最好顺序，也只有 95.69%，与简单“先全部获取再修复”相同。

结论：这批固定候选没有显示额外的最终 F1 调度空间，因此没有通过增加训练轮数继续推进。不同顺序的过程或成本仍可能不同，但不能据此声称已经得到实际节省。

10. 新 20 文档：来源规则有作用，但会误删

对应论文：IV-D 后半段；附录 D New Rule-Execution Test, [appendix.tex](#), 表 `tab:new_rule_feasibility`。

另取未参与之前生成试验的 20 个开发文档，40 次 Gemma 请求，39 个格式合格，共 407 条可编译候选。输入有 226 条参考事实及 28 个注入项。

策略	F1	参考保留率	注入移除	参考损失
不修复	94.02%	100.00%	0	0
两种策略全部获取再修复	96.63%	98.14%	17	4
仅 deletion	96.63%	98.14%	17	4
仅 augmentation	94.24%	100.00%	1	0

augmentation 修掉的那一项，deletion 也能修，因此它没有独有贡献。来源矛盾规则对 3 个注入项的移除确有必要，但也导致 2 条参考事实损失。

安全路径中最好的平均 F1 为 96.75%，只比最强固定策略高 0.12 点，低于冻结的 0.5 点标准。参考保留和双策略互补条件也未通过。

结论：本轮证明来源规则能改变编辑，却没有证明可靠、互补且值得学习调度的修复闭环。

附录的误删审计发现：4 条损失中，2 条来自过宽的类型禁止，2 条来自把“引用中出现两个实体”误判成“引用否定了它们的关系”。屏蔽全部删除依据能消除误删，也会消除全部修复，不能算已经解决问题。

11. 最新核验：完成了哪些，还缺哪些

对应论文：附录 D-A Rule Approval and Coverage, 表 `tab:rule_admission_coverage`；主文 IV-D 有简述。

新增入口要求规则与具体响应、原文、当前文档对和词表绑定。来源能定位之后，还要有对应批准，才允许规则参与获取和执行。

三组 × 八策略 × 十文档对，共 240 条离线回放：

准入方式	准入候选	F1	移除注入项	参考损失
只检查出处	407	96.63%	17	4
要求独立核验，目前无批准	0	94.02%	0	0
要求硬 schema，目前无可靠映射	0	94.02%	0	0

结论：程序能够隔离没有依据的规则，但当前没有有效覆盖。它不是“自动核验器已经提高修复效果”的实验。

自动模型核验另安排了 38 个非空包。唯一一次请求获 HTTP 200，但请求 Qwen3.8-27B-no-thinking 返回 Qwen/Qwen3.8-27B，按冻结的模型身份规则停止；37 个任务未派发。模型核验和联合核验两组未评估，不能写成零准确率。

51 项软件测试通过、400 条合成空判断回放通过，说明接口可检查，不增加真实模型实验样本。详细恢复条件见 本轮报告。

12. 其他辅助实验放在哪里

对应论文：附录 F Earlier Count-Penalty Component Study、H Further Repair Tests、I Rule Coverage and Time Records。

辅助实验	发现	不能拿它替代什么
旧 count 奖励下的组件研究	保存了旧策略和重训练消融	旧显著性不能直接搬到当前 rate 奖励
手写规则族	设计的 64 个缺陷全部检出	不能替代真实生成规则的执行成绩
SHACL-style 检查	所选 shapes 去重后不再删除其他三元组	不能说明 SHACL 语言表达不了更强约束
3,000 新闻标题的本地结构修复	结构分由 94.93 到 100.00	图中仍可能少了事实；不等于全事实恢复
45 标题真实 Qwen 抽取	类别准确率 0.556、关键词召回 0.178，结构分却为 100%	结构合法与语义准确是不同问题

这些实验有各自用途，但主 RL benchmark 仍是 450 条新闻构成的那张基础图。

13. 训练、调用和统计如何计数

对应论文：IV-E、附录 E Fixed Reward-Test Protocol、G Code and Result Files。

- 主奖励研究：40 个神经模型、10 个 ridge 模型，2,400 次预留场景评估。
- 当前 rate 消融：50 个新增神经模型，连同已有完整策略和启发式，共 2,100 次评估。
- 两组研究都在本地环境训练，模型 API 调用为 0。表里的 acquisition/calls 是模拟获取单位。
- 旧来源开发两轮有 80 次实际 Gemma 请求；新 20 文档批次另有 40 次。10-08 的核验另有一次 Qwen 请求，不能漏记。
- 缓存回放只使用已经取得的响应。策略在回放中少获取一包，不等于已经实测少花一次在线请求费用。

统计时先把同一个种子的 30 个场景求平均，再比较 10 个种子。DocRED 按 10 个文档对分析；不是把全部轨迹当成互相独立的样本。独立人工规则/编辑语义评价仍待作者安排。

14. 用这张表记住论文结论

对应论文：IV-F Discussion 与 Conclusion。

问题	当前答案
项目有没有真正训练 RL?	有，固定规则库上的 DDQN/DQN 已训练和测试
修正奖励是否有效?	相对旧 count 有明确改善
DDQN 是否已胜过强简单方法?	没有证明；启发式均值更高
哪个组件证据最清楚?	当前 rate 实验中的动作掩码
双策略候选是否有差异?	有，但等调用预算未显示数量优势
双策略是否带来安全独有修复?	当前开发实验尚未证明
真实生成规则已进入正式 RL 主实验吗?	尚未完成；已有固定流程、接口和开发诊断

问题	当前答案
自动核验是否解决了误删?	尚未证明; 当前严格入口没有修复覆盖

下一步重点是核实允许模型的身份配置, 完成限额核验, 判断可靠规则、策略互补与调度空间是否具备。详细待办见 [PROGRESS.md](#), 不是直接重复旧 RL 训练。